

OpenSearch Serverless: Vector Search by Hand

You will build a tiny vector index, embed five TravelMind FAQ snippets with Amazon Titan, and run a semantic kNN search — the same machinery a Bedrock Knowledge Base hides from you. Doing it by hand once is why the managed version stops feeling like magic.

The one rule. You did not create the collection. Your instructor made **one shared collection** for the whole class. You create your own index inside it, named `travelmind-<yourname>`. Nobody creates collections; that is the cost lever, and it stays locked.

The two locks (this is what you’ll trip on)

Reaching data in OpenSearch Serverless needs two independent grants. Managed OpenSearch has one. Serverless splits them, and a missing second lock looks identical to a missing first one: HTTP 403. Knowing which lock failed is the difference between a ten-second fix and a lost hour.

	Lock 1 — IAM gateway	Lock 2 — Data access policy
What it is	An IAM permission on your user	An OpenSearch-native policy on the collection
The grant	<code>aoss:APIAccessAll</code>	your ARN listed with index permissions
Set by	your IAM policy (BedrockLabOpenSearch)	the collection's data access policy
If it's missing	403 the moment you connect: <code>aoss:APIAccessAll</code>	403 only on create / read / write: no permissions for the resource

Both must open. IAM gets you to the door; the data access policy turns the key. In this lab your instructor owns both, so if you see a 403, read it and say which lock it is — that is half the exercise.

Run it

Where	Steps
CloudShell	<code>pip install opensearch-py boto3</code> · <code>export AOSS_HOST</code> and <code>MY_NAME</code> · <code>python opensearch_lab.py</code> (credentials are already there)
VS Code	<code>python -m venv .venv</code> and activate · <code>pip install opensearch-py boto3</code> · <code>aws configure</code> (or env keys) · set <code>AOSS_HOST</code> and <code>MY_NAME</code> · pick <code>.venv</code> as interpreter · run
Colab	<code>!pip install opensearch-py boto3</code> · set AWS keys via Colab secrets · set <code>AOSS_HOST</code> and <code>MY_NAME</code> · run the cells

Step 1 · Create your index

A vector index needs a `knn_vector` field. Two things must line up or nothing works: `index.knn` is on, and `dimension` equals what your embedding model returns. Titan Text V2 returns 1024.

```

MAPPING = {
    "settings": {"index.knn": True},
    "mappings": {"properties": {
        "embedding": {
            "type": "knn_vector",
            "dimension": 1024,          # must match your model
            "method": {"name": "hnsw", "engine": "faiss",
                "space_type": "l2"}},
        "text": {"type": "text"},
        "source": {"type": "keyword"}}}}
client.indices.create(index=INDEX, body=MAPPING)

```

```
INDEX = f"travelmind-{MY_NAME}" - your own index inside the shared collection
```

Step 2 · Authenticate, embed, ingest

Sign requests to the data endpoint, embed each snippet with Titan, and index it. The service name is the single most common mistake in this whole lab.

```

SERVICE = "aoss"          # serverless is "aoss", NOT "es"
creds = boto3.Session().get_credentials()
auth = AWSV4SignerAuth(creds, "us-east-1", SERVICE)
client = OpenSearch(
    hosts=[{"host": AOSS_HOST, "port": 443}], # bare host
    http_auth=auth, use_ssl=True,
    connection_class=RequestsHttpConnection)

def embed(text):
    body = json.dumps({"inputText": text,
        "dimensions": 1024, "normalize": True})
    r = brt.invoke_model(
        modelId="amazon.titan-embed-text-v2:0", body=body)
    return json.loads(r["body"].read())["embedding"]

```

```

DOCS = [
    ("Checked bags over 23 kg incur an excess fee.", "baggage"),
    ("Flexible fares allow free date changes.", "fares"),
    ("Lounge access is free for gold-tier members.", "loyalty"),
]

for text, source in DOCS:
    client.index(index=INDEX, body={"text": text,
        "source": source, "embedding": embed(text)})

time.sleep(30) # AOSS is near-real-time, not instant

```

Step 3 · Run a kNN search

Embed the question the same way, then ask for the k nearest vectors. The score ranks relevance; the baggage line should win.

```

qvec = embed("how much luggage can I bring?")
body = {
    "size": 3,
    "query": {"knn": {"embedding": {"vector": qvec, "k": 3}}},
    "_source": ["text", "source"]}
res = client.search(index=INDEX, body=body)
for h in res["hits"]["hits"]:
    print(h["_score"], h["_source"]["source"],
          h["_source"]["text"])

```

Expected, roughly:

```

0.81  baggage   Checked bags over 23 kg incur an excess fee.
0.39  fares     Flexible fares allow free date changes.
0.31  loyalty   Lounge access is free for gold-tier members.

```

When it 403s — read which lock

Symptom	Which lock / cause	Fix
403 mentioning aoss:APIAccessAll	Lock 1 — IAM gateway missing	attach BedrockLabOpenSearch (instructor)
403 on create or search, IAM looks fine	Lock 2 — your ARN not in the data access policy	instructor adds your ARN
403 / SignatureDoesNotMatch on every call	service set to “es”	use SERVICE = “aoss”
search returns 0 hits right after ingest	AOSS indexing lag	wait ~30s, retry the search
mapper_parsing / dimension error	mapping dimension ≠ embedding length	DIM must match the model (Titan V2 = 1024)
ConnectionError / cannot connect	host has https:// or a port	host = bare name only

What changes in production

- Swap the keys you minted for an IAM role (Lambda or ECS task role). Never ship access keys.
- service stays “aoss” for Serverless; “es” is for managed domains only.
- Data access policy: least privilege, split read vs write principals, and drop the class once the session ends.
- AllowFromPublic is a lab shortcut. Production uses a VPC or private network policy.
- Bulk-ingest with retries, and tune the HNSW method (ef_construction, m) for your recall and latency target.

Your turn

- Add two more FAQ snippets, re-run, and confirm they surface for a query that should match them.
- Drop k to 1 and ask something no document answers. What comes back, and why is the score low?
- Change space_type to cosinesimil, rebuild the index, and see whether the ranking moves.